

Student Project: Graph-Based Ergodic Coverage Under Dynamics

1 Project overview

The purpose of this project is to study graph-based algorithms for generating trajectories whose long-run occupation distribution matches a prescribed target distribution. The work is divided into three parts.

The first part considers the graph-optimization problem in isolation. The state space is discretized into cells, motion is represented by transitions on the cell-adjacency graph, and every visited cell contributes one discrete sample. This setting should be understood first because it contains the essential optimization ideas without introducing complications from continuous dynamics.

The second and main task is to execute the graph-generated paths with a continuous dynamical system. The physical time spent in cells is generally not equal to the number of discrete graph samples. This changes both the empirical occupation measure and the cost of moving between cells. The student will investigate how a graph algorithm should be adapted when its paths must be realized by first-order dynamics.

The final part introduces a more theoretical extension in which a state generates a sensor measure rather than a Dirac mass. The goal is to determine which state-distribution produces a desired expected sensor distribution. Two Gaussian examples provide explicit solutions and motivate a general transformation theory.

If the sensor measure depends on the agent's state variables, then the transformed occupation problem must generally be formulated on the agent's full state space rather than only on the physical domain. Consequently, the graph used to solve the transformed problem must also be lifted to this state space. Although this lifting can be computationally expensive, it is therefore a natural approach for the dynamic execution problem on the graph!

2 Part I: Discrete graph optimization

2.1 Intuition

Assume that a domain has been partitioned into finitely many cells. A trajectory is initially represented only by the sequence of cells that it visits. If a cell appears often in this sequence, then it receives a large empirical occupation mass. The optimization problem is to choose future cell visits so that this empirical distribution approaches a prescribed target distribution.

The algorithm operates in blocks. During one block, it first identifies an underrepresented target cell, then computes a graph path to that cell, and finally repeats the target cell as a self-loop until the block satisfies a contraction condition. The path provides spatial movement, while the repeated target samples provide occupation mass in a cell that is currently desirable.

The central example in this project is block-to-history contraction. Here, every new block is compared with the complete empirical history accumulated so far. Intuitively, the history describes what the trajectory has already done, and the new block is required to be better than that history under the currently frozen occupation error. This makes every block a corrective action relative to the full trajectory.

2.2 Common problem formulation

Let the physical domain X be partitioned into a finite collection of cells

$$\mathcal{C} = \{C_1, \dots, C_n\}.$$

The desired occupation distribution is denoted by

$$\mu = (\mu_1, \dots, \mu_n), \quad \mu_i \geq 0, \quad \sum_{i=1}^n \mu_i = 1.$$

Let N_i denote the number of samples assigned to cell C_i , and let

$$N = \sum_{i=1}^n N_i$$

be the total number of samples. The empirical occupation distribution is

$$\eta_i(N) = \frac{N_i}{N}.$$

The objective is to make $\eta(N)$ approach the desired distribution μ . The occupation error is measured by

$$G(\eta) = \|\eta - \mu\|_2^2 = \sum_{i=1}^n (\eta_i - \mu_i)^2.$$

The first variation, or gradient, of this objective is

$$g(\eta) = \nabla G(\eta) = 2(\eta - \mu).$$

A negative value of g_i indicates that cell C_i is underrepresented, while a positive value indicates that it is overrepresented.

Define

$$s = \min_{1 \leq i \leq n} g_i$$

and the history gap

$$H(\eta) = \langle g, \eta \rangle - s.$$

The history gap measures the difference between the gradient averaged under the current empirical occupation distribution and the smallest available gradient value.

2.3 Block-to-history algorithm

At the beginning of every block, the gradient is frozen. The algorithm then proceeds as follows.

Step 1: Compute the occupation deficit

Compute the current empirical measure η , objective $G(\eta)$, gradient g , minimum gradient value s , and history gap $H(\eta)$. Cells with values of g_i close to s are the most underrepresented and are therefore attractive future destinations.

Step 2: Select a suitable target cell

A simple rule is to choose a minimizer of the gradient,

$$d \in \arg \min_i g_i.$$

In practice, it is useful to balance occupation deficit and travel distance. First define the feasible destination set

$$\mathcal{D} = \{i : g_i - s \leq \rho_{\text{endpoint}} H(\eta)\},$$

where

$$0 \leq \rho_{\text{endpoint}} < \rho < 1.$$

Then choose

$$d \in \arg \min_{i \in \mathcal{D}} \left[w_{\text{dist}} \frac{\text{dist}(C_{\text{current}}, C_i)}{d_{\text{max}}} + w_{\text{def}} \frac{g_i - s}{\max\{H(\eta), \varepsilon\}} \right].$$

The first term favors nearby cells, while the second favors strongly underrepresented cells. The stricter endpoint factor ρ_{endpoint} leaves room for higher-error cells that may be encountered on the route.

Step 3: Use A^* to plan a route

Let the cell-adjacency graph be

$$\mathcal{G} = (V, E),$$

where each node represents one cell and neighboring cells are connected by an edge. A path from the current cell to the destination is

$$P = (i_0, i_1, \dots, i_m), \quad i_0 = i_{\text{current}}, \quad i_m = d.$$

For a destination d , define the positive excess error

$$\delta_i = \max\{0, g_i - g_d\}.$$

A practical graph edge cost for entering cell j is

$$c(i, j) = w_{\text{travel}} + w_{\text{error}} \frac{\delta_j}{\Delta g}, \quad \Delta g = \max_k g_k - \min_k g_k.$$

Thus, the actual accumulated path cost is

$$J(P) = \sum_{k=0}^{m-1} c(i_k, i_{k+1}).$$

It favors paths that are short and avoid strongly overrepresented intermediate cells.

For a rectangular grid, the standard heuristic is the Manhattan distance

$$h(i) = w_{\text{travel}} d_M(i, d),$$

where

$$d_M(i, d) = |r_i - r_d| + |c_i - c_d|.$$

Because the additional occupation penalty is nonnegative, this heuristic remains a lower bound for the standard graph cost and is admissible.

A^* expands nodes according to

$$f(i) = J(i) + h(i).$$

The result is a discrete path to the selected destination.

Step 4: Add destination self-loops until contraction

The already occupied start node is not counted again. The route contributes the new samples

$$(i_1, \dots, i_m).$$

If the route alone is insufficient, the destination d is appended repeatedly. Let p_{block} be the normalized counting measure of all samples generated during this block. The block is complete when

$$\langle g, p_{\text{block}} \rangle - s \leq \rho H(\eta).$$

Since g_d is close to s , additional destination samples reduce the average block error. A finite parameter `max-horizon` can limit the number of generated samples. If the cap is reached before contraction, the event should be recorded explicitly.

Step 5: Update the history

Append the block to the trajectory, update the cell counts, and compute

$$\eta_i^+ = \frac{N_i + N_i^{\text{block}}}{N + N^{\text{block}}}.$$

The gradient is then recomputed and the next block begins. The complete iteration is

$$\text{deficit} \longrightarrow \text{destination} \longrightarrow \text{A}^* \text{ route} \longrightarrow \text{self-loops until contraction} \longrightarrow \text{update}.$$

2.4 Block-to-block contraction

In addition to the block-to-history formulation, one can use a block-to-block contraction condition. Let p_k denote the normalized empirical measure of the previous block. After computing the new gradient g_k from the complete empirical history, the previous block is evaluated under this frozen gradient, which gives the warm block error

$$H_k^{\text{warm}} = \langle g_k, p_k \rangle - s_k, \quad s_k = \min_i g_{k,i}.$$

The next block p_{k+1} is then constructed to satisfy

$$\langle g_k, p_{k+1} \rangle - s_k \leq \rho H_k^{\text{warm}}.$$

A contraction argument of this form can be shown to yield convergence as well. The practical execution is essentially identical to the block-to-history algorithm described above: select a suitable destination cell, compute an A^* path to it, append destination self-loops until the contraction condition is satisfied, update the complete empirical measure, and recompute the gradient. The only substantive difference is the reference quantity used in the contraction test. Block-to-history compares the new block with the complete empirical history, whereas block-to-block compares it with the preceding block. Consequently, the same graph construction, A^* implementation, route-and-hold mechanism, and later dynamic realization can be reused almost verbatim.

3 Task: Execute the graph policy under dynamics

3.1 Why dynamics change the problem

The graph model treats every transition as one discrete sample. A physical system does not behave this way. It takes time to travel between cells, and the required time depends on the continuous state inside the current cell.

For example, suppose two adjacent cells are equally desirable according to the graph gradient. One may be reached almost immediately because the agent is already close to their common boundary. Reaching the other may require crossing most of the current cell. Although both actions correspond to one graph edge, their physical occupation effects are different.

More generally, a graph path

$$(i_0, i_1, \dots, i_m)$$

does not produce an equal-weight occupation measure. It produces a residence-time vector determined by the dynamics and the continuous trajectory. Time spent travelling through source and intermediate cells contributes to the empirical occupation, even when those cells were not selected as destinations.

This leads to two separate research questions:

1. How should a discrete graph path be executed and evaluated under dynamics?
2. Should physical residence time also be fed back into destination selection, A^* , and the contraction condition?

The first question should be answered before altering the graph optimizer. This preserves a clean separation between graph optimization and trajectory execution.

3.2 First-order dynamics

Consider

$$\dot{x}(t) = u(t), \quad \|u(t)\| \leq u_{\max}, \quad x(t) \in X \subset \mathbb{R}^2.$$

Suppose $x \in C_i$ and the next graph node is the adjacent cell C_j . Choose an entry waypoint slightly inside C_j ,

$$x_j^{\text{entry}} = \Pi_{C_j}^\varepsilon(x).$$

The minimum travel time is

$$t(i, j) = \frac{\|x_j^{\text{entry}} - x\|}{u_{\max}}.$$

The segment crosses the boundary between C_i and C_j , so split the transition time as

$$t(i, j) = t_i(i, j) + t_j(i, j),$$

where $t_i(i, j)$ is assigned to the source cell and $t_j(i, j)$ to the destination cell.

For a complete route, summing these contributions gives a transit residence vector

$$r = (r_1, \dots, r_n), \quad T_r = \sum_{i=1}^n r_i.$$

The physical empirical measure is

$$\eta_i(T) = \frac{T_i}{T},$$

where T_i is the total physical time spent in C_i and

$$T = \sum_i T_i.$$

The physical trajectory objective is still

$$G(\eta(T)) = \|\eta(T) - \mu\|_2^2.$$

3.3 Executing the holding phase as a patrol

A destination self-loop need not be implemented by setting $u = 0$. Instead, the agent may patrol within the destination cell for the prescribed duration. For a rectangular cell, choose random interior waypoints and move between them at speed at most $u_{\max}$. Because the cell is convex, the complete line segment between two interior points remains inside the cell.

The final patrol segment should be truncated so that the total patrol duration is exactly τ_{req} . All patrol time is then assigned to the destination cell. This produces the same cell-level occupation update as a stationary hold while generating a nontrivial continuous trajectory.

The student should compare:

1. stationary holding with $u = 0$;
2. uniform within-cell waypoint patrol;
3. bounded patrol followed by gradient recomputation.

For the bounded version, define

$$\tau_{\text{exec}} = \min\{\tau_{\text{req}}, \tau_{\max}\}.$$

If the timer is reached before contraction, update the physical occupation and recompute the gradient. This is a physical release-and-hold mechanism and prevents one frozen gradient from controlling an excessively long destination residence phase.

3.4 Dynamics-aware A^*

The standard A^* path is computed entirely on the graph. A further extension is to incorporate predicted physical residence time into the path cost. For destination d , let

$$\delta_i = \max\{0, g_i - g_d\}.$$

For a transition $i \rightarrow j$, define

$$c_x(i, j) = t_i(i, j)\delta_i + t_j(i, j)\delta_j.$$

This is the occupation error accumulated during the physical transition.

Since the transition residence times depend on the continuous entry position, two paths entering the same cell from different sides need not have the same future cost. A practical A^* state is therefore

$$q = (i, i_{\text{prev}}),$$

with an associated continuous entry waypoint x_q . For every successor, the algorithm computes the next entry waypoint, transition times, and residence-time edge cost.

A dynamics-scaled Manhattan heuristic is

$$h(i) = w_h \frac{d_M(i, d) \min\{c_w, c_h\}}{u_{\max}} \Delta g, \quad \Delta g = \max_k g_k - \min_k g_k.$$

4 Final task: Transformation of sensor measures

4.1 General problem

Let $(\Omega, \Sigma_\Omega, \eta)$ and (U, Σ_U, μ) be probability spaces. For every state $\omega \in \Omega$, let

$$\xi(\omega, \cdot) \in \mathcal{M}(U)$$

be the sensor measure generated at that state. The objective is to find a state distribution η such that the expected sensor measure equals the desired target measure:

$$\mathbb{E}_{\omega \sim \eta} [\xi(\omega, \cdot)] = \mu.$$

If

$$\xi(\omega, \cdot) = f(\omega, \cdot)\mu,$$

then the condition is equivalent to

$$\mathbb{E}_{\omega \sim \eta} [f(\omega, u)] = 1 \quad \text{for } \mu\text{-almost every } u \in U.$$

The general task is to determine when such an η exists, whether it is unique, and how it can be approximated numerically.

4.2 Example 1: Gaussian position sensors

Let

$$U = \Omega = \mathbb{R}^d, \quad \mu = \mathcal{N}(m, \Sigma_\mu),$$

and suppose the sensor measure at state ω is

$$\xi(\omega, \cdot) = \mathcal{N}(\omega, \Sigma_\sigma).$$

The desired condition becomes the convolution equation

$$\int_{\mathbb{R}^d} f_{\Sigma_\sigma}(s - \omega) \eta(d\omega) = f_{\Sigma_\mu}(s - m).$$

Taking Fourier transforms gives

$$\widehat{\eta}(\zeta) \widehat{f}_{\Sigma_\sigma}(\zeta) = e^{-i\zeta^\top m} \widehat{f}_{\Sigma_\mu}(\zeta).$$

If

$$\Sigma_\mu - \Sigma_\sigma \succeq 0,$$

then a solution is

$$\eta = \mathcal{N}(m, \Sigma_\mu - \Sigma_\sigma).$$

Thus, the desired state distribution is less diffuse than the target sensor distribution because each state already generates a Gaussian sensing footprint.

4.3 Example 2: Position and orientation

Let

$$\Omega = \mathbb{R}^2 \times [0, 2\pi)$$

and write a state as $\omega = (p, \theta)$. Consider the directional sensor density

$$\xi((p, \theta), s) = \frac{1}{N} f_{\Sigma_\sigma}(s - p) \exp(\kappa \cos(\text{atan2}(s - p) - \theta)),$$

where

$$N = 2\pi I_0(\kappa).$$

If the desired target μ has no preferred orientation, the angular contribution must disappear after averaging. A natural solution is therefore uniform in orientation:

$$\theta \sim \mathcal{U}[0, 2\pi).$$

Indeed,

$$\int_0^{2\pi} \xi((p, \theta), s) d\theta = f_{\Sigma_\sigma}(s - p).$$

The remaining position problem reduces to the Gaussian convolution example. When the covariance condition holds, this suggests

$$\eta = \mathcal{N}(m, \Sigma_\mu - \Sigma_\sigma) \otimes \mathcal{U}[0, 2\pi).$$

4.4 Generalization task and connection to dynamics

The student should generalize these examples by studying the operator

$$K\eta = \int_{\Omega} \xi(\omega, \cdot) \eta(d\omega)$$

and the inverse problem

$$K\eta = \mu.$$

The main questions are:

1. Under which conditions does a solution exist?
2. When is the solution unique?
3. How can an approximate solution be computed when no exact solution exists?
4. Which properties of the original ergodic problem are preserved by the transformation?
5. How do finite-time trajectory solutions change when Dirac sensors are replaced by smooth sensor measures?

The position-orientation example requires a graph in a higher-dimensional state space, for example a graph over

$$(x, y, \theta).$$

This is computationally more expensive than a position-only grid, but it creates a direct connection to dynamics. Orientation can represent a physical state or motion constraint, and transitions on the higher-dimensional graph can encode which position-orientation changes are dynamically feasible.

One possible final experiment is therefore to construct an indicator or cell-based graph on $\mathbb{R}^2 \times S^1$, solve the transformed occupation problem there, and execute the resulting graph path with orientation-dependent dynamics. The method may be slow because the number of graph nodes grows rapidly with dimension, but it provides a concrete bridge between sensor-measure transformations and dynamics-aware graph execution.